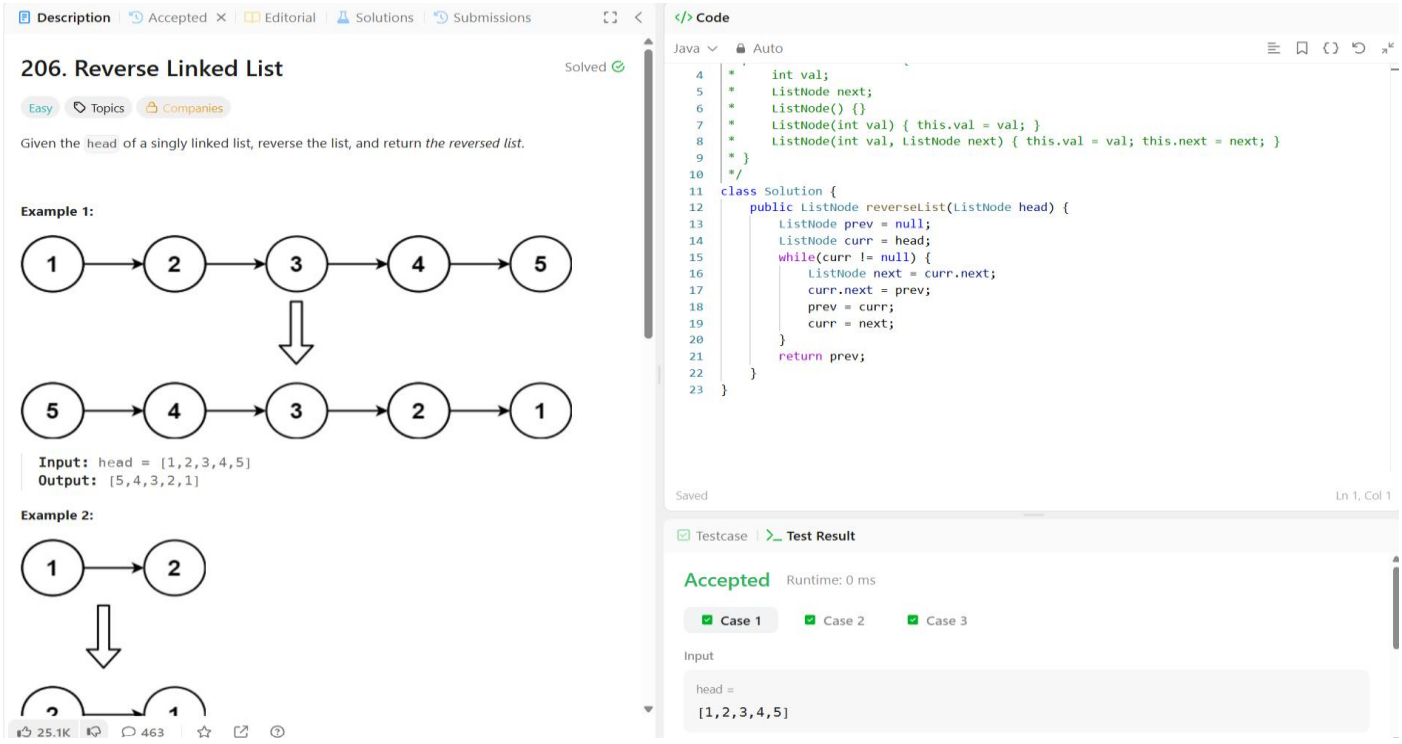
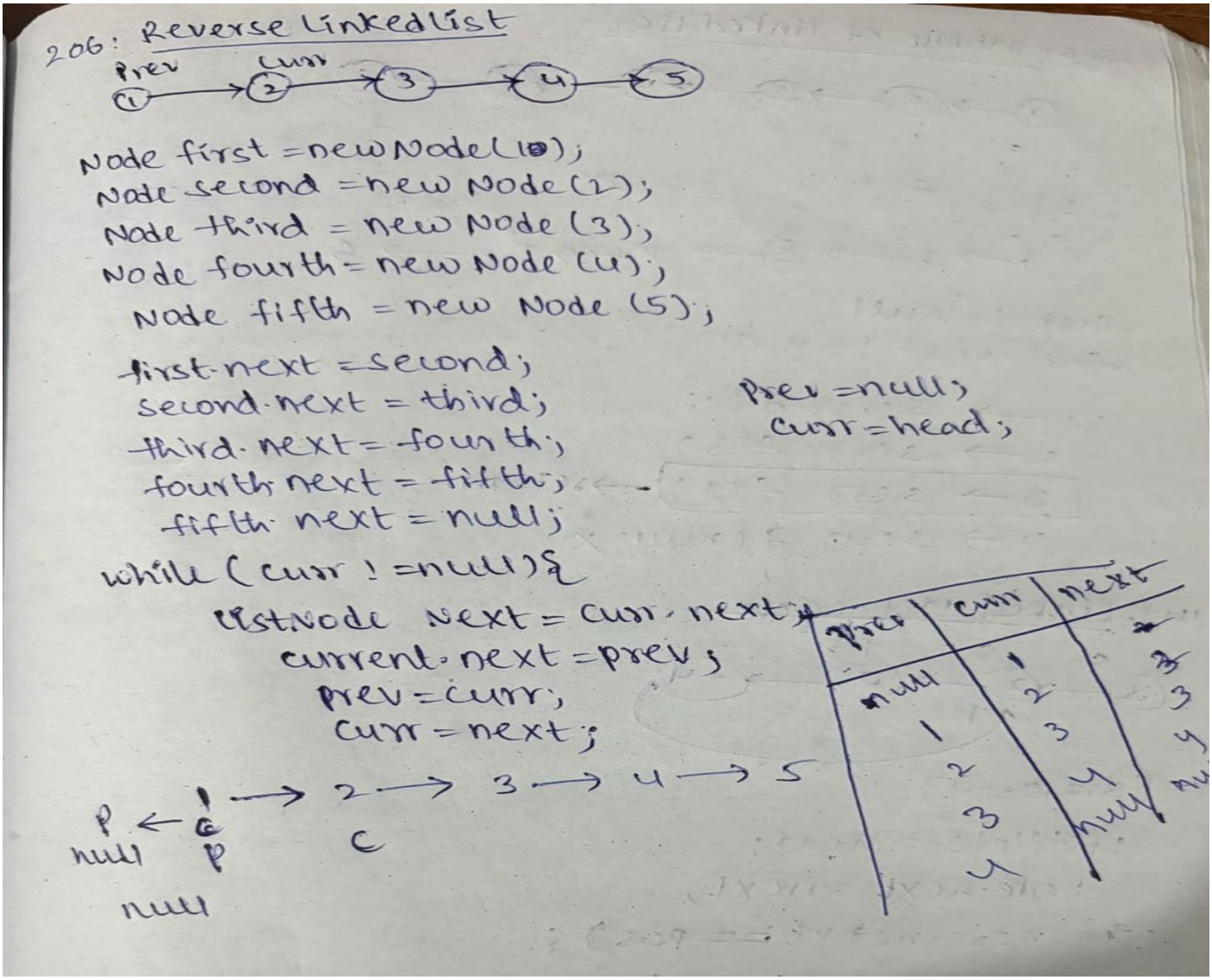
M. Siri Chandana

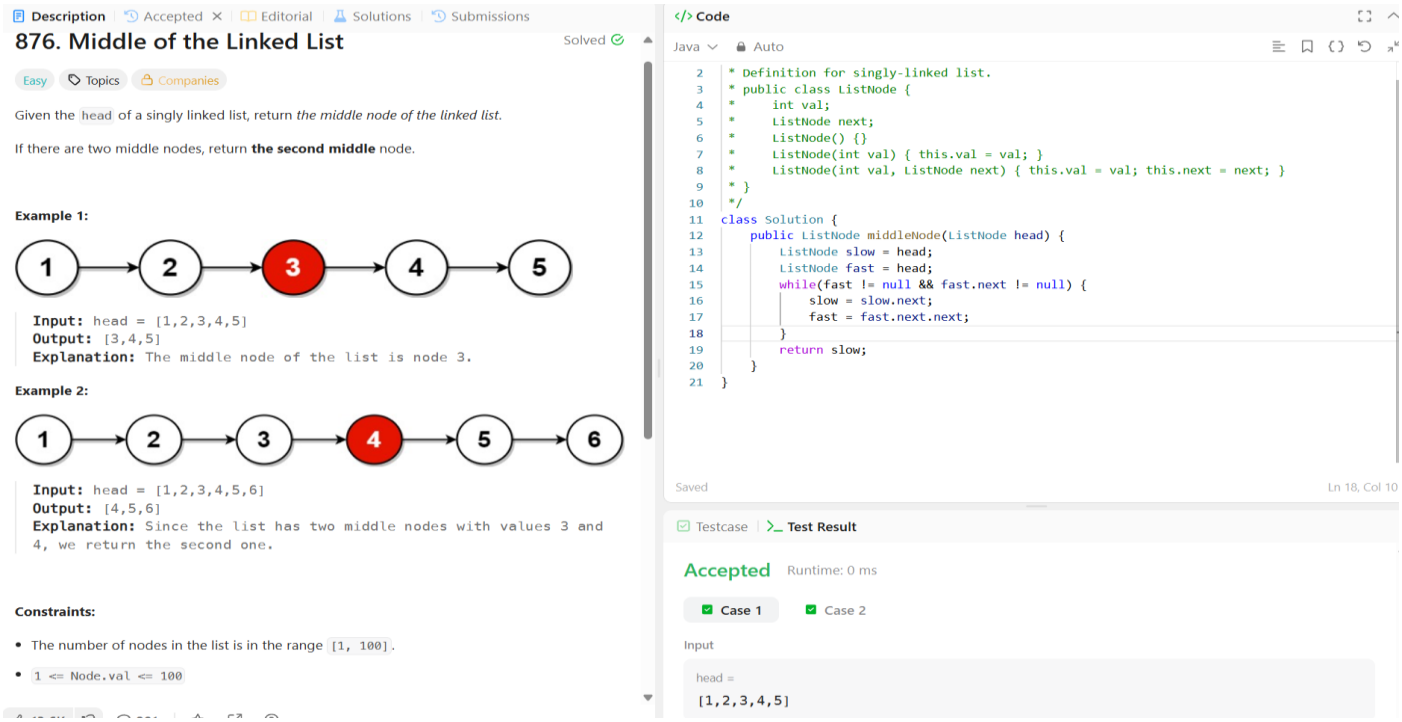
2405A41139

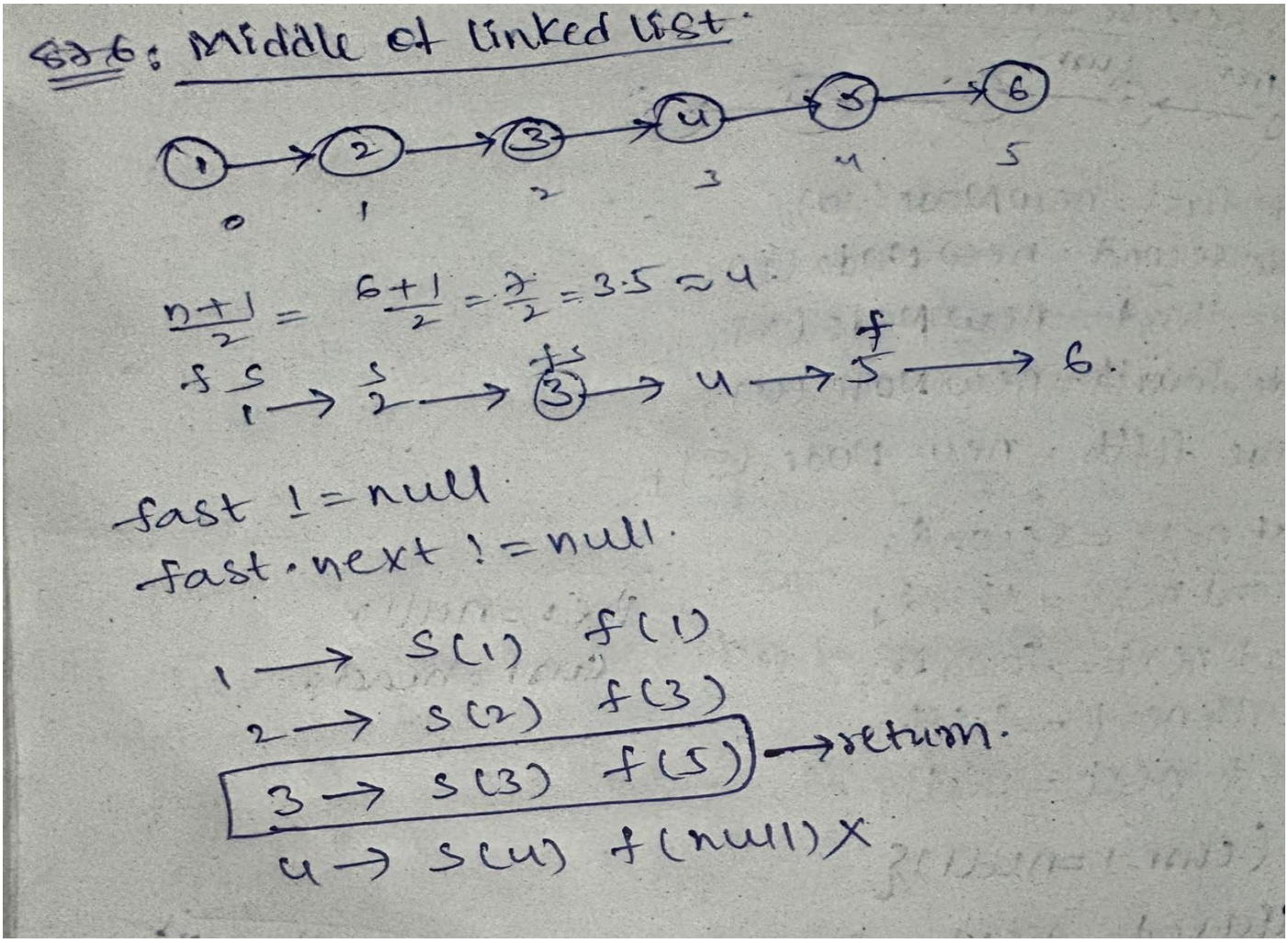
1.Reverse Linked List - LC206





2. Middle of the Linked List – LC 876





3. Linked List Cycle – LC 141

141. Linked List Cycle

Solved

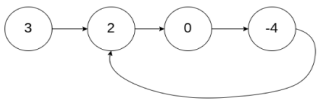
Easy | Topics | Companies

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter**.

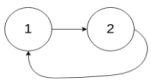
Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail
connects to the 1st node (0-indexed).
```

Example 2:




```
Input: head = [1,2], pos = 0
Output: true
Explanation: There is a cycle in the linked list, where the tail
connects to the 0th node.
```

18K | 584

Java | Auto

```java
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while(fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if(slow == fast) {
                return true;
            }
        }

        return false;
```

Saved

Testcase | Test Result

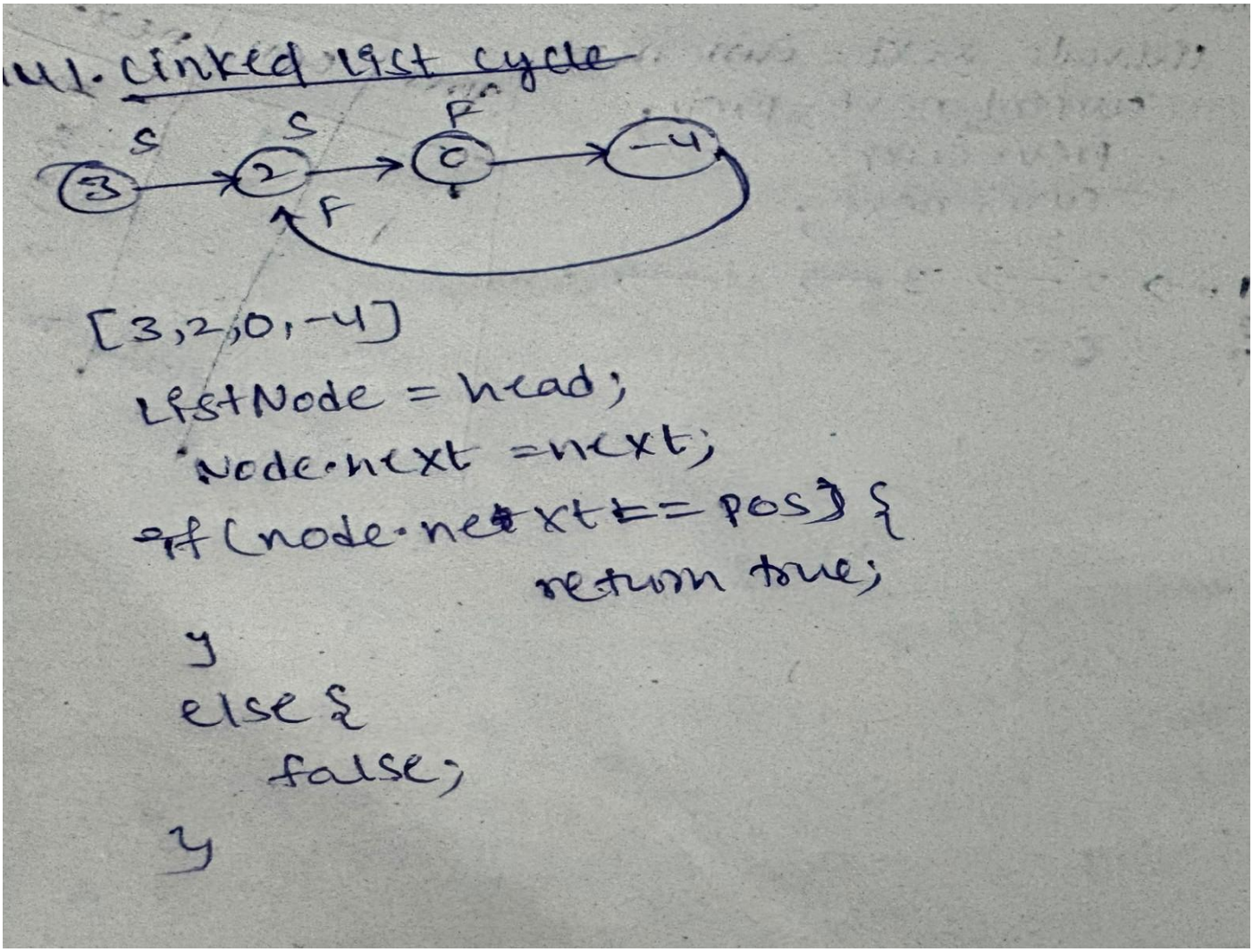
Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =

[3,2,0,-4]



4. Merge Two Sorted Lists – LC 21

Description | Accepted | Editorial | Solutions | Submissions

21. Merge Two Sorted Lists

Solved

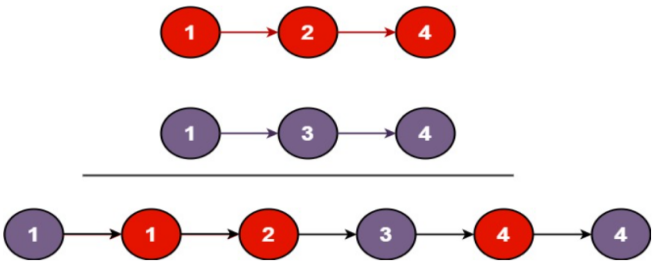
Easy | Topics | Companies

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list.*

Example 1:




```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

Example 2:

```
Input: list1 = [], list2 = []
Output: []
```

25.6K | 666

</> Code

Java | Auto

```
*     int val;
*     ListNode next;
*     ListNode() {}
*     ListNode(int val) { this.val = val; }
*     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
* }
*/
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode dummy = new ListNode(-1);
        ListNode temp = dummy;
        while(list1 != null && list2 != null) {
            if(list1.val <= list2.val) {
                temp.next = list1;
                list1 = list1.next;
            } else {
                temp.next = list2;
                list2 = list2.next;
            }

            temp = temp.next;
        }
        if(list1 != null) {
            temp.next = list1;
        }
```

Saved

Testcase | Test Result

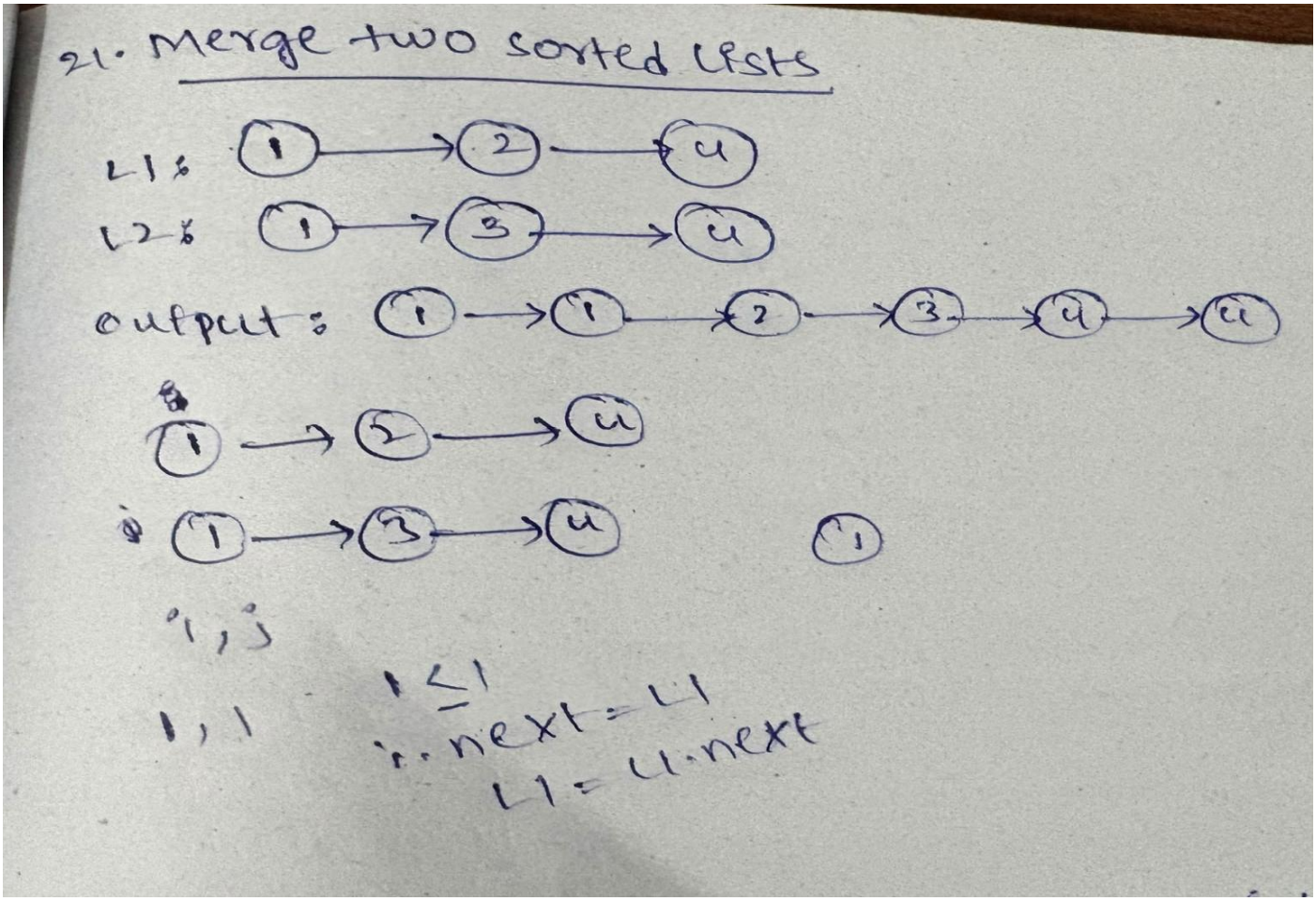
Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

list1 =

[1,2,4]



5. Remove Nth Node From End of List – LC 19

Description | Accepted | Editorial | Solutions | Submissions

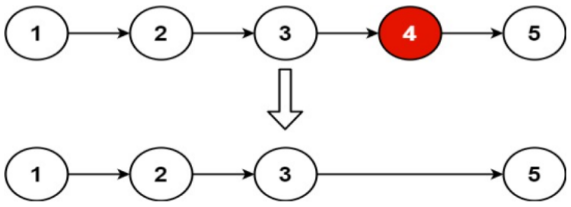
19. Remove Nth Node From End of List

Solved

Medium | Topics | Companies | Hint

Given the `head` of a linked list, remove the `n`th node from the end of the list and return its head.

Example 1:




```
Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]
```

Example 2:

```
Input: head = [1], n = 1
Output: []
```

Example 3:

```
Input: head = [1,2], n = 1
Output: [1]
```

21.8K | 406

Code

Java | Auto

```
/**
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode slow = head;
        ListNode fast = head;
        for(int i = 0; i < n; i++) {
            fast = fast.next;
        }
        if(fast == null) {
            return head.next;
        }
        while(fast.next != null) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return head;
    }
```

Saved | Ln 25, Col

Testcase | Test Result

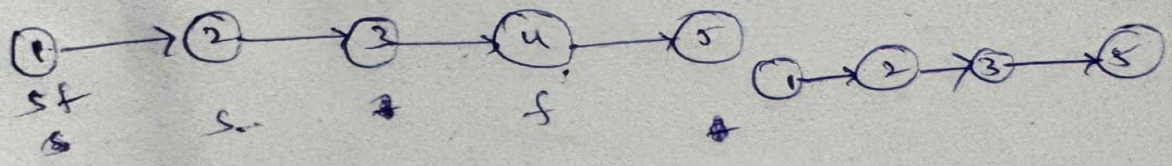
Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =
[1,2,3,4,5]

19. Remove Nth Node from the end of the List



fast = fast.next
slow = slow.next

1 s-1,1
2 f-2,1
3 3,2
4 4,3
5 5,4